

Failure (force the parser to actually fail) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the exact classes and mechanism), **Advanced** (what a production `except` clause actually needs to account for). Read Basic first even if you're comfortable with Pydantic — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Forcing a real mismatch, and finding the exact error class](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Forcing a real mismatch, and finding the exact error class

Basic Version

This is the previous exercise's failure case, but on purpose and closely inspected. You need a Pydantic shape with a field that's genuinely hard to fill from a certain kind of question — like a number field, asked about something that isn't a number at all.

Run the chain, catch whatever error comes back, and don't just print it — print its exact Python type. That's the part your code would need to `except` in real life.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The goal here is to identify the *exact* exception class `.with_structured_output(...)` raises when the model's reply can't be coerced into your Pydantic shape, so your production code can catch that specific class — not a broad `Exception`.

Design a Pydantic model with a required numeric field (`int` or `float`), and ask about something purely descriptive that has no clean number in it. This forces a genuine mismatch, rather than relying on the model happening to misbehave.

Here are the exact pieces you need: - A Pydantic model with a number field: `class Rating(BaseModel): score: int` - A structured chain: `ChatOpenAI().with_structured_output(Rating)` - A question with no number in it: `"Describe your favorite color."` - Catch it, then print `type(e)` and `type(e).__mro__` (its full class hierarchy) — this tells you exactly which `except SomeSpecificError:` clause would catch it, and which broader classes it also happens to be an instance of.

Run this and write down the exact class name before moving to Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

A required `int` field combined with a prompt that has no number-shaped answer at all is a reliable way to force a mismatch — an *optional* field or a vaguely-numeric prompt might let the model guess something that technically parses, and you'd get a flaky test instead of a reliable one.

The harder real-world question: LangChain's version, and the underlying model's exact wording, can change which class actually gets raised — sometimes it's `pydantic.ValidationError` directly, sometimes a LangChain-specific wrapper around it. Code that catches only `ValidationError` by name, written once and never revisited, can silently stop catching anything the day a library upgrade changes that wrapping.

The design answer: catch the specific class you've confirmed, but always keep a broader `except Exception` fallback that logs the *actual* class it caught (`type(e).__name__`) — not just a generic message. That way, if the exact type changes under you, your logs tell you immediately instead of the error silently falling through uncaught.

Difference between Basic, Intermediate, and Advanced: Basic tells you to force a mismatch and inspect the error's type. Intermediate gives you the exact mechanism (`type(e).__mro__`) and the exact pieces to force it reliably. Advanced explains why the specific class you find today isn't guaranteed to stay the same across library versions, and gives you the defensive pattern (specific catch + logged fallback) that survives that instability.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a shape with a required number field called score

build a structured chain using that shape

ask it something with no number in the answer at all

catch whatever error happens
    print exactly what kind of error it is
```

Here is almost the whole thing:

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Rating(BaseModel):
    score: int

structured_model = ChatOpenAI().with_structured_output(Rating)
```

What's missing: the `try/except` call itself, printing the exact error type, and a one-line note on which `except` clause you'd actually write in real code. Add those, then check the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
class Rating(BaseModel):
    score: int

structured_model = ChatOpenAI(model="...", temperature=0).with_structured_output(

try:
    structured_model.invoke("Describe your favorite color in a full sentence, no
except Exception as e:
    print(type(e))           # exact class
    print(type(e).__mro__)   # its full parent chain
    print(str(e))           # the actual message
```

If this particular prompt doesn't fail (the model might still invent a number), try a more clearly non-numeric prompt, or a stricter field type (like a `Literal[...]` with a fixed set of choices that clearly excludes the answer).

What's missing to finish: the actual `except <SpecificClass>:` clause once you know what to write, replacing the broad `except Exception:`. Write it yourself, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Write the version you'd actually ship, not just the version that demonstrates the failure: wrap the chain in a function, catch the specific class with a real recovery path (not just a print), and keep the broad fallback that logs the actual class it caught.

```
def get_rating(question: str) -> "Rating | None":
    try:
        return structured_model.invoke(question)
    except ValidationError as e:
        # expected case: log and return None, let the caller decide what to do
        ...
    except Exception as e:
        # unexpected case: log the real class so you notice if it ever changes
        ...
```

Fill in both `except` bodies yourself — think about what a caller of `get_rating()` should actually get back in each case (a default value? a re-raised, more specific error of your own?) — then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic gets you a script that demonstrates the failure and prints its type. Intermediate turns that into the actual `except <SpecificClass>:` clause once you've identified it. Advanced asks the production question underneath both — what should a *caller* of this function actually receive when the expected failure happens, versus an unexpected one — which is the difference between a script that shows you an error and a function you'd trust in a real app.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)